

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE Nº XX

NOMBRE COMPLETO: José Eduardo Martínez García

Nº de Cuenta: 320304679

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 31/10/2026

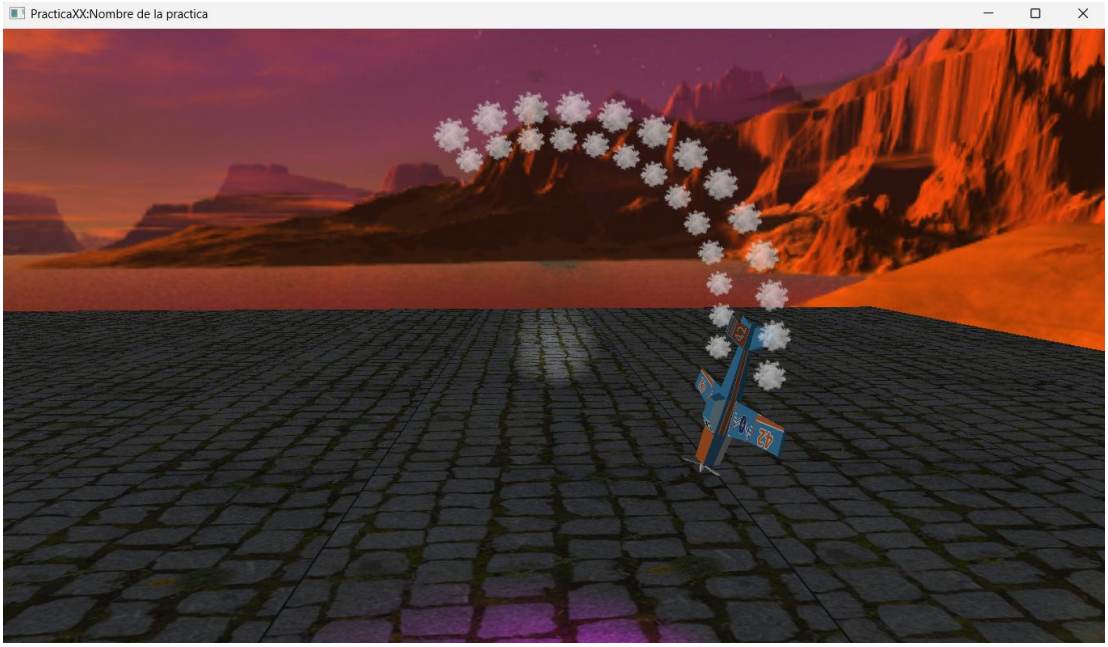
CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas

a) Avión haciendo acrobacias o vuelo a partir de funciones y algoritmos

Lo que hice fue armar un recorrido completo para el avión usando solo funciones matemáticas, sin guardar puntos a mano. El avión sale de un borde del piso y mezcla rectas con ondas, giros horizontales, seis vueltas verticales, dos ascensos a 45° y una subida pronunciada con caída, hasta regresar al mismo punto donde empezó. La punta del avión siempre apunta hacia donde se mueve.



Recorrido y control del tiempo

```
const float DURACION_TRAMO = 5.0f;
const TramoVuelo recorridoAvion[] = {
    {RECTA_AVION,60}, {LOOP_AVION,0}, {RECTA_AVION,60}, {LOOP_AVION,0},
    {GIRO_AVION,360}, {SUBIDA_AVION,90}, {LOOP_AVION,0},
    {RECTA_AVION,60}, {LOOP_AVION,0}, {CAIDA_AVION,100}, {GIRO_AVION,180},
    {RECTA_AVION,60}, {LOOP_AVION,0}, {SUBIDA_AVION,90}, {GIRO_AVION,360},
    {RECTA_AVION,60}, {LOOP_AVION,0}, {RECTA_AVION,160}, {GIRO_AVION,180}
};

float paso = fmod(tiempo / DURACION_TRAMO, float(TRAMOS_AVION));
int actual = int(paso);
float u = paso - actual;
glm::vec3 posicion(0.0f, 20.0f, -300.0f);
```

$$q = \left(\frac{t}{5}\right) \bmod 19; \quad n = \lfloor q \rfloor; \quad u = q - n$$

El arreglo dice qué maniobra sigue y en qué orden: en rectas y ascensos el segundo valor es la distancia, en los giros es el ángulo. Cada tramo dura 5 segundos, así que el recorrido de 19 tramos

se repite cada 95 segundos. El parámetro u va de 0 a 1 dentro del tramo actual, y el avión empieza en (0, 20, −300).

Giros horizontales (las curvas)

```
float a = tramo.distancia * toRadians * t;
posicion += glm::vec3(20*(cos(yaw)-cos(yaw+a)), 0,
   20*(sin(yaw+a)-sin(yaw)));
yaw += a;
```

$$a = A\pi u \,/\, 180$$

$$\Delta x = 20\,[cos\,\psi - cos(\psi + a)] \qquad \Delta z = 20\,[sin(\psi + a) - sin\,\psi] \qquad \psi \leftarrow \psi + a$$

Aquí A es el ángulo del tramo en grados y ψ es la dirección con la que el avión llega a la curva. El radio es de 20 unidades. Es un arco de círculo: con 360° regresa al mismo punto y misma dirección, y con 180° queda volteado para hacer el regreso.

Vueltas verticales

```
glm::vec2 vueltaVertical(float u)
{
   float a = 2.0f * PI_AVION * u;
   return glm::vec2(12.0f * sin(a),
      12.0f * (1.0f - cos(a)));
}
```

$$a = 2\pi u \qquad s(u) = 12\,sen\,a \qquad h(u) = 12\,(1 - cos\,a)$$

Es un círculo de radio 12 puesto de forma vertical. s es lo que avanza en la dirección del vuelo y h es lo que sube. Cuando u llega a 1 los dos valores vuelven a cero, entonces el avión da la vuelta completa y sigue por la misma línea. Esta misma función se usa en las seis vueltas del recorrido.

Ondas en las rectas

```
if (tramo.tipo == LOOP_AVION) return vueltaVertical(u);
if (tramo.tipo == SUBIDA_AVION)
   return glm::vec2(tramo.distancia*u, subidaAvion(u));
if (tramo.tipo == CAIDA_AVION) return caidaAvion(u);
float onda = sin(PI_AVION*u);
return glm::vec2(tramo.distancia*u,
   1.2f*onda*onda*sin(4.0f*PI_AVION*u));
```

$$s(u) = Du \qquad h(u) = 1.2\,sen^2(\pi u)\,sen(4\pi u)$$

Esta función decide qué perfil (avance y altura) usa cada tipo de tramo. En las rectas, D es la distancia y se le agrega una ondulación: $sen(4\pi u)$ da dos oscilaciones y sen²(πu) las apaga en los extremos para que no haya saltos al pasar al siguiente tramo. El 1.2 es la amplitud.

Ubicación del perfil en la escena

```
glm::vec2 perfil = perfilVuelo(tramo, t);
```

```
posicion += glm::vec3(sin(yaw)*perfil.x, perfil.y,
   cos(yaw)*perfil.x);
```

$$\Delta x = s\,sen\,\psi \qquad \Delta y = h \qquad \Delta z = s\,cos\,\psi$$

Las funciones de arriba solo dan avance y altura. Con la dirección ψ del avión se pasan a los ejes de la escena, por eso la misma vuelta o la misma onda sirve tanto en el trayecto de ida como en el de regreso. Esto se suma tramo por tramo desde el inicio hasta el tramo actual.

Ascensos de 45 grados

```
float x = 90.0f * u;
float altura = 0.0f;
const float limites[] = {0, 4, 22, 26, 64, 68, 86, 90};
const float pendientes[] = {0, 1, 1, 0, 0, -1, -1, 0};
for (int i = 0; i < 7; ++i)
{
   float largo = limites[i + 1] - limites[i];
   float t = glm::clamp((x - limites[i]) / largo, 0.0f, 1.0f);
   altura += largo * (pendientes[i] * t +
      (pendientes[i + 1] - pendientes[i]) *
      (t * 0.5f - sin(PI_AVION * t) / (2.0f * PI_AVION)));
}
```

Los 90 de avance se dividen en 7 partes con los límites b y sus pendientes p. La pendiente pasa de 0 a 1 (sube a 45°), se mantiene, regresa a 0 para volar arriba y luego se repite con −1 para bajar. La funicon definida hace que el cambio de pendiente sea suave, sin esquinas. Termina 22 unidades arriba de la altura base, se mantiene ahí entre x = 26 y x = 64, y regresa a 0.

Subida pronunciada con keyframes (Hermite)

```
const glm::vec2 puntos[] = {{0,0}, {28,68}, {48,24}, {100,0}};
const glm::vec2 tangentes[] = {{45,0}, {6,0}, {30,-40}, {60,0}};
float paso = glm::clamp(u, 0.0f, 1.0f) * 3.0f;
int i = glm::min(int(paso), 2);
float t = paso - i;
float t2 = t * t, t3 = t2 * t;
return (2*t3-3*t2+1)*puntos[i] + (t3-2*t2+t)*tangentes[i] +
   (-2*t3+3*t2)*puntos[i+1] + (t3-t2)*tangentes[i+1];
```

$$H(t) = (2t^3 - 3t^2 + 1)\,P_{\mathrm{i}} + (t^3 - 2t^2 + t)\,T_{\mathrm{i}} + (-2t^3 + 3t^2)\,P_{\mathrm{i+1}} + (t^3 - t^2)\,T_{\mathrm{i+1}}$$

Aquí uso la idea de keyframes: cuatro puntos P (avance, altura) unidos con una curva cúbica de Hermite, donde las tangentes T controlan la dirección y la rapidez. Los tres intervalos duran lo mismo y t va de 0 a 1 en cada uno. La cima sube 68 unidades sobre la altura base, la maniobra termina 100 unidades adelante y solo aparece una vez por recorrido.

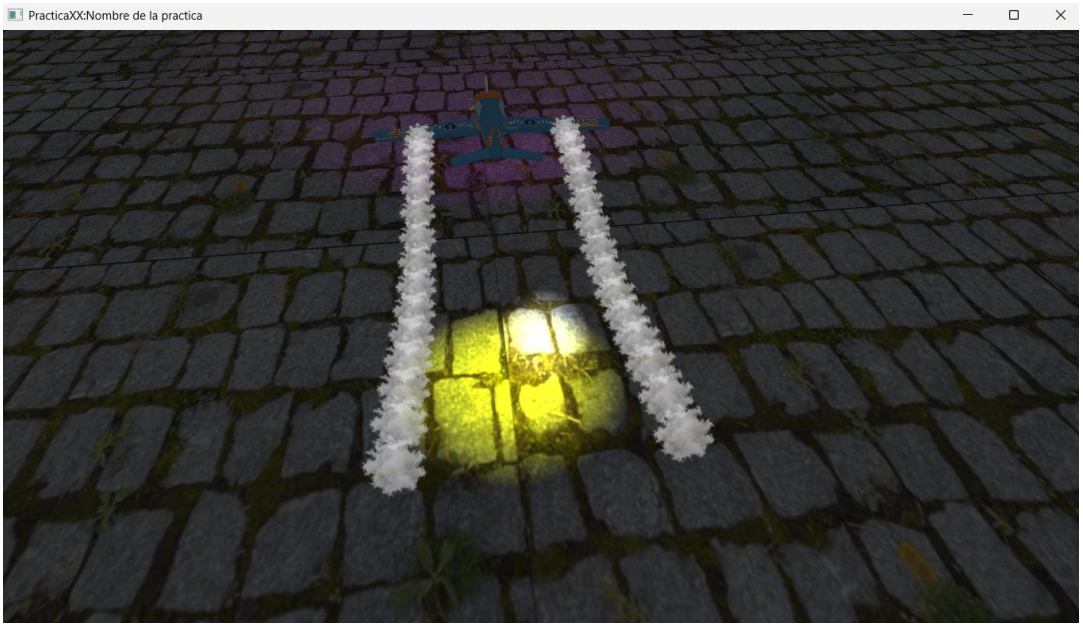
Llamadas en el ciclo principal

```
float segundosAvion = now - lastTime;
...
actualizaAvion(segundosAvion);
lucesAvion();
...
glUniformMatrix4fv(uniformModel,1,GL_FALSE,glm::value_ptr(modelAvionAux));
avion_texturizado.RenderModel();
glUniformMatrix4fv(uniformModel,1,GL_FALSE,glm::value_ptr(modelHelice));
helice_avion.RenderModel();
```

En cada cuadro se actualiza el avión con el tiempo real entre cuadros (segundosAvion), separado del deltaTime que ya usaba la cámara. Después se dibujan el avión y la hélice con sus propias matrices.

b) Añadir que el avión vaya dejando humo en su camino, que aparezca desde sus motores y desaparezca después de 2 segundos

Para el humo puse dos emisores, uno en cada ala. Cada 0.15 segundos sale una pareja de nubes con una textura de humo con fondo transparente. La nube se queda donde salió, sube poco a poco y deja una estela detrás del avión. A los 2 segundos deja de dibujarse.



Partículas de humo

```
const float VIDA_HUMO = 2.0f;
const float INTERVALO_HUMO = 0.15f;
const int MAX_HUMO = 2 * (int(VIDA_HUMO / INTERVALO_HUMO) + 1);
struct ParticulaHumo { glm::vec3 posicion; float edad = VIDA_HUMO; };
ParticulaHumo humo[MAX_HUMO];
int siguienteHumo = 0;
```

$$MAX_HUMO = 2\left(\left\lfloor\frac{VIDA}{INTERVALO}\right\rfloor + 1\right) = 2\left(\left\lfloor\frac{2}{0.15}\right\rfloor + 1\right) = 28$$

Cada nube guarda su posición y su edad. Todas empiezan con la edad igual a la vida, o sea que arrancan apagadas. El arreglo es circular y su tamaño sale de calcular cuántas nubes caben vivas al mismo tiempo (14 parejas, 28 espacios). Si se cambia INTERVALO_HUMO se separan más o menos las nubes.

Salida del humo desde los motores

```
for (auto& particula : humo) particula.edad += dt;
emisionHumo += dt;
while (emisionHumo >= INTERVALO_HUMO)
{
   emisionHumo -= INTERVALO_HUMO;
   if (emisionHumo >= VIDA_HUMO) continue;
   glm::mat4 emisor = matrizAvion(tiempoAvion - emisionHumo);
   for (int ala = -1; ala <= 1; ala += 2)
   {
      ParticulaHumo& particula = humo[siguienteHumo];
      particula.posicion = glm::vec3(emisor * glm::vec4(0.5f,0.0f,ala*4.5f,1.0f));
      particula.edad = emisionHumo;
      siguienteHumo = (siguienteHumo + 1) % MAX_HUMO;
   }
}
```

Textura, transparencia y dibujo del humo

```
humoTexture = Texture("Textures/humo.png");
humoTexture.LoadTextureA();
...
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA,GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_FALSE);
for (auto& particula : humo)
   if (particula.edad < VIDA_HUMO) visibles.push_back(&particula);
std::sort(visibles.begin(),visibles.end(),[&](ParticulaHumo* a, ParticulaHumo* b) {
   return (view*glm::vec4(a->posicion,1)).z < (view*glm::vec4(b->posicion,1)).z;
});
for (auto particula : visibles)
{
   glm::mat4 model = glm::transpose(glm::mat4(glm::mat3(view)));
   glm::vec3 posicion = particula->posicion + glm::vec3(0,0.7f*particula->edad,0);
   model[3] = glm::vec4(posicion,1);
   model = glm::scale(model,glm::vec3(2.8f));
   glUniform1f(opacidad,0.65f);
   meshList[7]->RenderMeshModel();
}
glDepthMask(GL_TRUE);
```

Shader del humo

```
// Vertex
uv = tex;
gl_Position = projection * view * model * vec4(pos,1);
// Fragment
vec4 t = texture(theTexture, uv);
color = vec4(t.rgb, t.a * opacidad);
```

Para el humo hice un shader aparte y muy sencillo, sin luces, es decir, solo toma el color de la textura y multiplica su alfa por la opacidad. Se guarda en shaderList[1] y se usa nada más al dibujar las nubes, al final de todo.

2. Problemas presentados

- Al unir los tramos el avión daba saltos de posición o de altura en los cambios de maniobra. Se arregló haciendo que cada tramo empezara y terminara en pendiente cero (la onda se apaga en los extremos con $sen^2(\pi u)$) y sumando los desplazamientos de todos los tramos desde el inicio, en lugar de reiniciar la posición en cada uno.
- El avión no regresaba exactamente al punto de salida y cada vuelta se iba desviando. Se resolvió usando fmod con el tiempo total y calculando la posición completa en cada cuadro, sin acumular incrementos. Con los ángulos de los giros sumando 360° por pares de 180° y 360° el recorrido cierra solo y forma una trayectoria ciclicla.
- Las nubes se veían de lado o desaparecían según el ángulo de la cámara. Se arregló haciendo que cada cuadrado mirara siempre a la cámara con la transpuesta de la rotación de la vista, y con un shader propio que no depende de las luces ni de las normales.

3. Conclusión

Me parecieron de dificultad media. Lo que más costó fue juntar tantas maniobras sin que el avión se moviera de golpe, y para eso ayudó mucho dividir todo en tramos y revisar uno por uno. Con seno y coseno salieron las vueltas, los giros y las ondas, y con los keyframes se logró una subida más marcada con pocos puntos. El humo fue más sencillo una vez entendido que el emisor va pegado al avión pero la nube ya emitida se queda en el mundo.